

Design Rationale

REQ 2: Be wary of dung (beetle)

A. GoldenBeetle Implementation

Design 1: Create GoldenBeetle class extending Creature abstract class and implement Eatable and Producible interfaces

Pros	Cons
Promotes code reusability	Requires maintenance for extra class and interfaces
Clear modularisation of responsibilities using interfaces and abstract classes	Results in complex program if every functionality needs abstract classes and interfaces
Enhances code readability as the abstract class and interfaces define a clear contract of what is expected for GoldenBeetle class	Connascence of Name – any changes in the signatures of methods canProduce(), produce(), eat(), allowableActions() requires modification in GoldenBeetle class

Design 2: Implement all logic directly inside the GoldenBeetle class

Pros	Cons
Enhances code maintainability as no additional classes or interfaces need to be maintained	GoldenBeetle class takes on multiple responsibilities
	Existing code need modification if new behaviours are added

B. Following Behaviour

Design 1: Directly implement getAction() and distance() methods in GoldenBeetle class

Pros	Cons
Enhances code maintainability as no additional classes or interfaces need to be used	Class is responsible for both its behaviour and following logic
Simpler implementation if only GoldenBeetle can follow an actor	Duplicated code will be implemented across every actor with the ability to follow a followable actor

Design 2: Create a FollowBehaviour class

Pros	Cons
Promotes code reusability as other actors can easily reuse this class	Requires maintenance of an extra class
Allows runtime modification	Connascence of Meaning – Modification needed if the meaning and implementation of the behaviour change
GoldenBeetle is loosely couple to the following logic by simply adding FollowBehaviour to its map	

C. GoldenEgg Implementation

Design 1: Create GoldenEgg class extending Item abstract class

Pros	Cons
Simpler implementation if there is only one type of egg	Violates Liskov Substitution Principle – Golden Egg may not behave like an Item whenever an Item is expected

Design 2: Create GoldenEgg class extending Egg abstract class

Pros	Cons
Provides a clear contract of common methods and attributes that should be implemented by an Egg	Limits the development of functionality as only 1 inheritance is allowed per class
GoldenEgg behaves like an Egg and is substitutable whenever an Egg is expected	Requires maintenance of an extra class

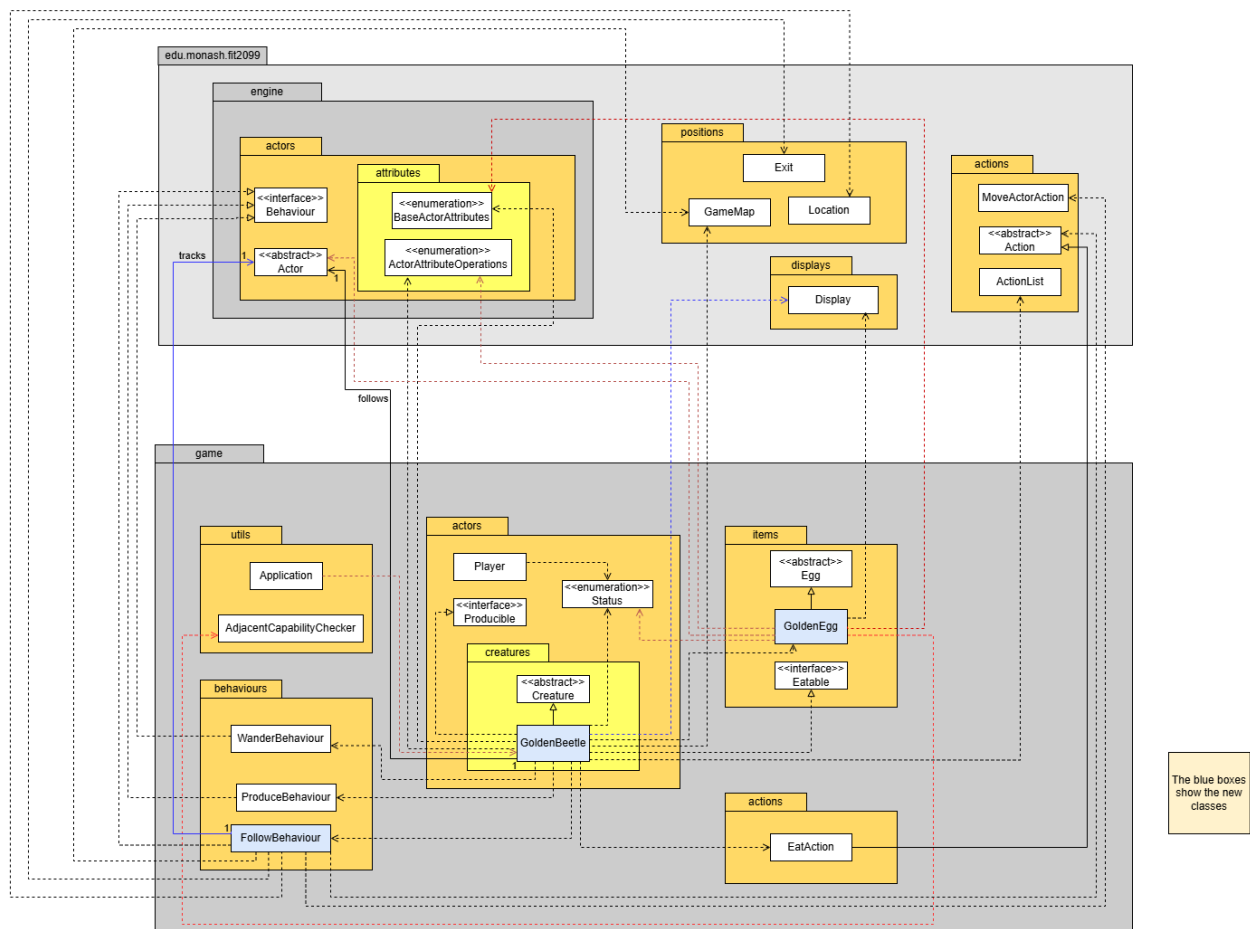
D. Hatching Condition of GoldenEgg

Design 1: Use 'instanceof' and downcasting to check the type of ground that GoldenEgg can on, then return true or false based on the type of ground identified

Pros	Cons
Simpler and direct implementation	Modification needed when a new cursed entity is added
	Violates Liskov Substitution Principle - Requires checking of type of ground using 'instanceof'

Design 2: Override shouldHatch() method to allow GoldenEgg to hatch nearby CURSED status

Pros	Cons
Enhance code extensibility as the status can be added to future cursed entity that allow GoldenEgg to be hatched nearby it	Additional status logic implementation in the ground classes needed
Enhance code readability	



The diagram above shows the final design of REQ2, which utilizes Design 1 for GoldenBeetle implementation, Design 2 for following behaviour, GoldenEgg Implementation and hatching condition check of GoldenEgg.

The GoldenBeetle class extends the abstract Creature class and GoldenEgg class extends the abstract Egg class. The new concrete classes inherit the methods and attributes from their existing abstract classes. We abstract these identities to avoid repetition in code (DRY) as these classes share many same attributes and methods with the abstract classes they extend from. The new creature or item created solely use the constructor of their super class with different initialized values, enabling them to replace the usage of their subclasses in code (Liskov Substitution Principle).

In the GoldenBeetle class, the allowableActions() is overridden to add an eat action and follow behaviour when it's nearby an Followable actor, giving GoldenBeetle class the responsibility of adding an eat action to the options, reducing dependencies of other classes on this class but segregating this responsibility (Single Responsibility Principle). Besides, GoldenBeetle class to rely on abstraction rather than concrete implementation, adhering Dependency Inversion Principle. Moreover, using different interfaces to define different traits (i.e. can produce) and abilities (i.e. ability to spawn and be eaten) of the concrete GoldenBeetle class allows us to separate different responsibilities to different interfaces, resulting in better maintainability of the system (Single Responsibility Principle). This implementation allows the GoldenBeetle's traits to be separated by their own purposes (Interface Segregation Principle). Future actions and behaviours can simply be added or implemented by the GoldenBeetle class, without changing the existing classes, adhering to the Open Closed Principle.

Adding a new FollowBehaviour allows us to segregate the logic of tracking the movement of an actor and moving following the target from the GoldenBeetle class itself, rather embedding the following logic directly in the GoldenBeetle class by implementing getAction() and distance(). Using this implementation, the GoldenBeetle class will be responsible for its own purpose only, where different responsibilities are distributed to different classes (Single Responsibility Principle). As this design adheres the Open Closed Principle, there will be minimal impact on the existing class if any new requirement for the following behaviour arises. The priority of behaviours is managed using an order-based TreeMap.

GoldenEgg class extends the abstract Egg class and override the hatch(), shouldHatch() and createHatching() methods to implement hatching behaviours. By extending Egg instead of the more general Item class, GoldenEgg is tied to the specific characteristics and behaviours associated with an Egg rather than the general Item. Connascence of Algorithm is demonstrated in this design where general egg-related logic is consistent across different egg types but unique hatching behaviours are managed by GoldenEgg subclass. If new requirements for the GoldenEgg hatching condition arise, this design ensures minimal impact on the existing systems by adhering to the Single Responsibility Principle as the GoldenEgg class handles its specific logic without interfering with Item or Egg classes.

GoldenEgg class will depend on Status enum class, which acts as an indicator to allow GoldenEgg to be hatched when it is nearby cursed entities. Using the dependency on the Status enum class, we avoided using downcasting to checking for the types of the

concrete classes that are fine to be hatched on, resulting in easier functionality extension without modifying existing code with additional if-else checks (Open-Closed Principle).